EN3150 Assignment 03

Resource-Constrained CNN for Edge Image Classification

[Student Name 1] ([**Index Number 1**])
[Student Name 2] ([**Index Number 2**])

GitHub repository: **[https://github.com/your-org/CNN-for-image-classification]**

October 10, 2026

1 Data Preparation

1.1 Dataset

This work uses the RGB variant of the **EuroSAT** land-use/land-cover dataset, which consists of 27,000 Sentinel-2 satellite image patches of size 64×64 pixels across 10 balanced-ish classes: AnnualCrop, Forest, HerbaceousVegetation, Highway, Industrial, Pasture, PermanentCrop, Residential, River, and SeaLake. Because the images are already at 64×64 resolution, no downscaling was required to satisfy the assignment's maximum-resolution constraint for the custom models; the two pretrained models used in Section 5 were given a larger input (discussed there) since ImageNet-pretrained filters are substantially less effective at 64×64.

1.2 Train / Validation / Test split

The dataset was split **70% / 15% / 15%** into training, validation, and test sets using a stratified split (per-class proportions preserved) with a fixed random seed (42) for reproducibility. This gives approximately 18,900 training, 4,050 validation, and 4,050 test images; exact per-class counts vary by at most one or two images due to stratified rounding.

1.3 Preprocessing and augmentation

All images were normalized per channel. For the two custom models (Section 2), pixel values were normalized with mean = std = 0.5 per channel (mapping to the $[-1, 1]$ range), and the training split was augmented with random horizontal and vertical flips (satellite imagery has no canonical orientation, so both flips are valid label-preserving augmentations). The validation and test splits use the same resizing and normalization but no augmentation. For the pretrained models, ImageNet normalization statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$) were used instead, matching what those networks were originally trained with.

[Confirm that the dataset information sheet referenced in the assignment brief has been filled in by the group.]

2 Custom Architecture Design

Both custom models follow the standard CNN pipeline: `[Conv + ReLU]` → `Pool` → `[Conv + ReLU]` → `Pool` → ... → `Classifier` → `Softmax`, taking a $3 \times 64 \times 64$ RGB input and producing a 10-

class prediction.

2.1 Model A: Standard CNN

Model A interleaves regular 2D convolutions with max-pooling layers, followed by a flatten and two fully-connected layers.

Table 1: Model A architecture.

Layer	Configuration	Output shape
Input	–	$3 \times 64 \times 64$
Conv2D + ReLU	16 filters, 3×3, pad 1	$16 \times 64 \times 64$
MaxPool2D	2×2	$16 \times 32 \times 32$
Conv2D + ReLU	32 filters, 3×3, pad 1	$32 \times 32 \times 32$
MaxPool2D	2×2	$32 \times 16 \times 16$
Conv2D + ReLU	64 filters, 3×3, pad 1	$64 \times 16 \times 16$
MaxPool2D	2×2	$64 \times 8 \times 8$
Flatten	–	4096
Linear + ReLU	$4096 \to 128$	128
Linear	$128 \to 10$	10

Hand-calculated parameter count. For a standard Conv2D layer the trainable parameter count is $C_{out} \cdot C_{in} \cdot K^2 + C_{out}$ (the $+C_{out}$ term is the bias), and for a Linear layer it is $\text{in} \cdot \text{out} + \text{out}$.

Table 2: Model A parameter breakdown.

Layer	Formula	Parameters
Conv1 ($3 \to 16$)	$16 \cdot 3 \cdot 3^2 + 16$	448
Conv2 ($16 \to 32$)	$32 \cdot 16 \cdot 3^2 + 32$	4,640
Conv3 ($32 \to 64$)	$64 \cdot 32 \cdot 3^2 + 64$	18,496
FC1 ($4096 \to 128$)	$4096 \cdot 128 + 128$	524,416
FC2 ($128 \to 10$)	$128 \cdot 10 + 10$	1,290
Total		**549,290**

Note that FC1 alone accounts for $524{,}416/549{,}290 \approx 95.5\%$ of the model's parameters: flattening an $8 \times 8 \times 64$ feature map into a 128-unit fully-connected layer is by far the most expensive operation in this architecture, far more costly than any of the convolutional layers. This observation motivates Model B's design (global average pooling instead of a large flatten + FC head). The hand-calculated total of 549,290 was cross-checked against a runtime parameter count (`sum(p.numel() for p in model.parameters())`) and matches exactly.

2.2 Model B: Lightweight depthwise-separable CNN

Model B replaces every standard convolution with a **depthwise separable convolution**: a depthwise $K \times K$ convolution applied independently per input channel (no cross-channel mixing), followed by a 1×1 pointwise convolution that mixes channels. This is the construction used in MobileNet-style architectures, and it reduces the multiply-accumulate (MAC) cost of a 3×3 convolution by

roughly a factor of $K^2 \cdot C_{out}/(K^2 + C_{out})$ (around 8–9× for typical layer widths) relative to a full standard convolution of the same shape. Each depthwise-separable block is followed by batch normalization and ReLU. The classifier head uses global average pooling (GAP) instead of a flatten, which is the main reason Model B's parameter count stays small.

Table 3: Model B architecture.

Layer	Configuration	Output shape
Input	–	$3 \times 64 \times 64$
DWSepConv + BN + ReLU	$3 \to 32$, 3×3	$32 \times 64 \times 64$
MaxPool2D	2×2	$32 \times 32 \times 32$
DWSepConv + BN + ReLU	$32 \to 64$, 3×3	$64 \times 32 \times 32$
MaxPool2D	2×2	$64 \times 16 \times 16$
DWSepConv + BN + ReLU	$64 \to 128$, 3×3	$128 \times 16 \times 16$
MaxPool2D	2×2	$128 \times 8 \times 8$
AdaptiveAvgPool2D	output 1×1	$128 \times 1 \times 1$
Linear	$128 \to 10$	10

Hand-calculated parameter count. A depthwise conv (groups $= C_{in}$, one filter per channel) costs $C_{in} \cdot K^2 + C_{in}$; a pointwise 1×1 conv costs $C_{in} \cdot C_{out} + C_{out}$; and a BatchNorm2D layer contributes $2C$ learnable parameters (scale γ and shift β only – the running mean/variance are buffers, not trainable parameters).

Table 4: Model B parameter breakdown.

Layer	Formula	Parameters
DWSepConv1 depthwise (3 ch)	$3 \cdot 3^2 + 3$	30
DWSepConv1 pointwise ($3 \to 32$)	$3 \cdot 32 + 32$	128
DWSepConv1 BatchNorm (32 ch)	$2 \cdot 32$	64
DWSepConv2 depthwise (32 ch)	$32 \cdot 3^2 + 32$	320
DWSepConv2 pointwise ($32 \to 64$)	$32 \cdot 64 + 64$	2,112
DWSepConv2 BatchNorm (64 ch)	$2 \cdot 64$	128
DWSepConv3 depthwise (64 ch)	$64 \cdot 3^2 + 64$	640
DWSepConv3 pointwise ($64 \to 128$)	$64 \cdot 128 + 128$	8,320
DWSepConv3 BatchNorm (128 ch)	$2 \cdot 128$	256
Classifier ($128 \to 10$)	$128 \cdot 10 + 10$	1,290
Total		**13,288**

This is well within the assignment's 100,000-parameter budget for Model B, using only 13.3% of it (86.7% headroom). This headroom is a deliberate consequence of the GAP-based head rather than an accident: a flatten of the final $8 \times 8 \times 128$ feature map into even a modestly-sized FC layer would alone cost well over 100,000 parameters, so GAP was essential to the design, not an afterthought. The hand-calculated total was likewise cross-checked against a runtime parameter count and matches exactly.

2.3 Activation function justification

Both models use **ReLU** ($f(x) = \max(0, x)$) exclusively as the nonlinearity, rather than sigmoid or tanh. This is justified on hardware-efficiency grounds, which matters directly for the resource-constrained/edge deployment target of this assignment:

- ReLU requires only a single comparison and a conditional select per element, with no transcendental (exponential) function evaluation. Sigmoid and tanh both require computing e^{-x}, which is substantially more expensive on microcontroller-class hardware that lacks a dedicated floating-point transcendental unit.
- ReLU's gradient is either 0 or 1, which is cheap to compute during backpropagation and avoids the vanishing-gradient problem that sigmoid/tanh exhibit for large $|x|$.
- ReLU activations are naturally sparse (exactly zero for negative inputs), which many inference engines and accelerators can exploit to skip computation.

3 Optimizer Selection and Tuning

Adam with a learning rate of 1×10^{-3} was selected as the optimizer for the final training runs of both custom models. Adam was chosen because it adapts the effective learning rate per parameter using running estimates of the first and second gradient moments, which typically gives faster and more reliable convergence than plain SGD without requiring extensive learning-rate tuning – a useful property given the limited CPU-only compute budget available for this assignment.

3.1 Comparison against SGD and SGD with Momentum

Two independent experiments were run to justify this choice.

Model A, 10-epoch sweep. Adam ($\text{lr} = 10^{-3}$), plain SGD ($\text{lr} = 10^{-2}$), and SGD with momentum ($\text{lr} = 10^{-2}$, momentum $= 0.9$) were each trained for 10 epochs. SGD and SGD+momentum were given the same (larger) learning rate since, without adaptive scaling, SGD needs a bigger step to make comparable progress in the same number of epochs; momentum's effect can then be isolated cleanly since the learning rate is held fixed between the two SGD variants.

Table 5: Model A optimizer sweep, validation accuracy/loss after 10 epochs.

Optimizer	LR	Train acc.	Val. acc.	Val. loss
Adam	10^{-3}	87.0%	83.1%	0.484
SGD	10^{-2}	62.2%	46.5%	1.520
SGD + Momentum (0.9)	10^{-2}	87.2%	83.7%	0.484

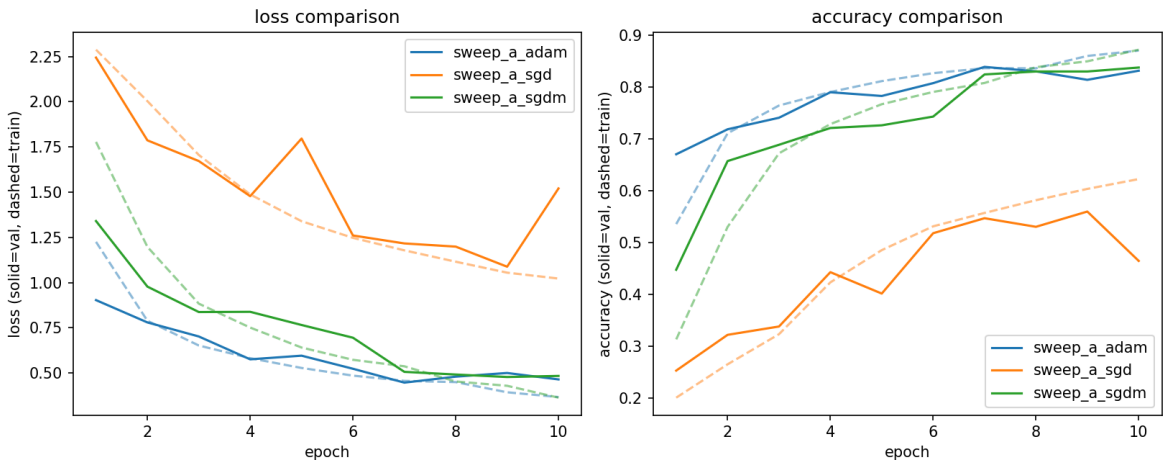


Figure 1: Model A optimizer sweep: validation (solid) and training (dashed) loss and accuracy vs. epoch, for Adam, plain SGD, and SGD with momentum.

Model B, full 20-epoch runs. As a second, independent confirmation, Model B was trained to completion (20 epochs, test-set evaluated) under all three optimizer settings.

Table 6: Model B optimizer comparison, full 20-epoch test-set results.

Optimizer	Test accuracy	Macro precision	Macro recall
Adam	**89.76%**	0.8948	0.8963
SGD	82.35%	0.8163	0.8161
SGD + Momentum (0.9)	89.41%	0.8924	0.8913

Discussion: effect of momentum. In both experiments, plain SGD is clearly the weakest optimizer – its validation loss is both higher and visibly less stable (Figure 1 shows a sharp spike around epoch 5), and on Model B it is nearly 7.5 percentage points behind Adam in test accuracy. Adding momentum (velocity term $v_t = \mu v_{t-1} + \nabla_\theta \mathcal{L}$, with the parameter update using v_t instead of the raw gradient) at the *same* learning rate dramatically changes this picture: the momentum term accumulates an exponentially-weighted average of past gradients, which (i) damps oscillation in directions where the gradient sign repeatedly flips, and (ii) effectively increases the step size in directions where the gradient is consistently pointing the same way. This is exactly what is observed here – SGD with momentum converges almost as fast as Adam and reaches a nearly identical final accuracy (within 0.35 percentage points on Model B, and ahead of Adam on the Model A sweep by epoch 10), despite using a much simpler update rule. This confirms that most of the practical convergence benefit over plain SGD in this setting comes from momentum specifically, rather than from Adam's additional per-parameter adaptive scaling.

4 Custom Model Training and Evaluation

Both Model A and Model B were trained for 20 epochs using Adam (lr $= 10^{-3}$), as justified in Section 3.

4.1 Loss curves

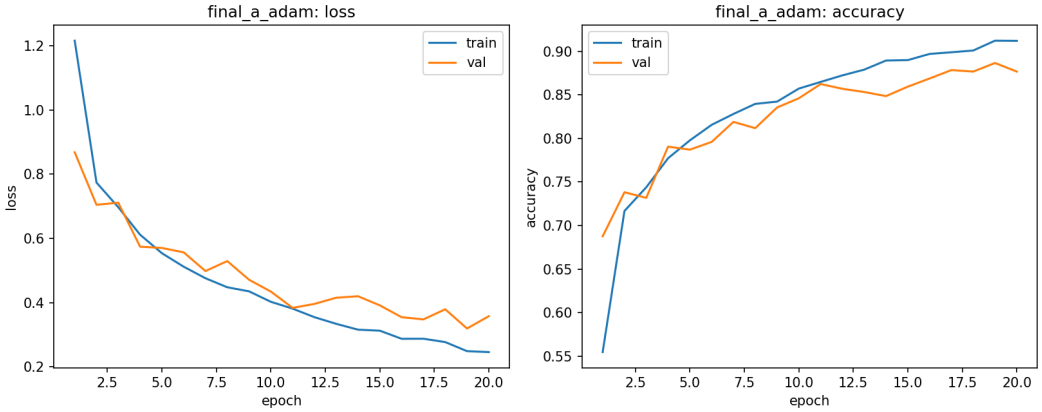


Figure 2: Model A training/validation loss and accuracy vs. epoch.

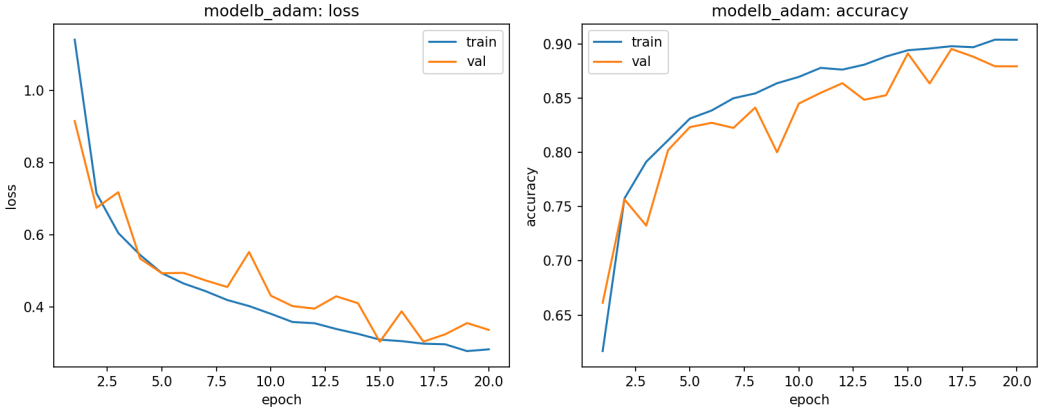


Figure 3: Model B training/validation loss and accuracy vs. epoch.

Model A's training and validation curves (Figure 2) track closely for roughly the first 11 epochs, after which a gap opens: by epoch 20, training accuracy reaches 91.2% while validation accuracy plateaus and becomes noisy around 85–89%. This is a mild overfitting signal, consistent with the architectural observation from Section 2 that Model A's 524,416-parameter FC1 layer (95.5% of all parameters) has no dropout, weight decay, or batch normalization to regularize it. Model B's curves (Figure 3) are comparatively noisier epoch-to-epoch but do not show the same systematic late-training divergence, plausibly because its batch normalization layers provide some regularization and because it simply has far fewer parameters available to overfit with.

4.2 Confusion matrices and per-class metrics

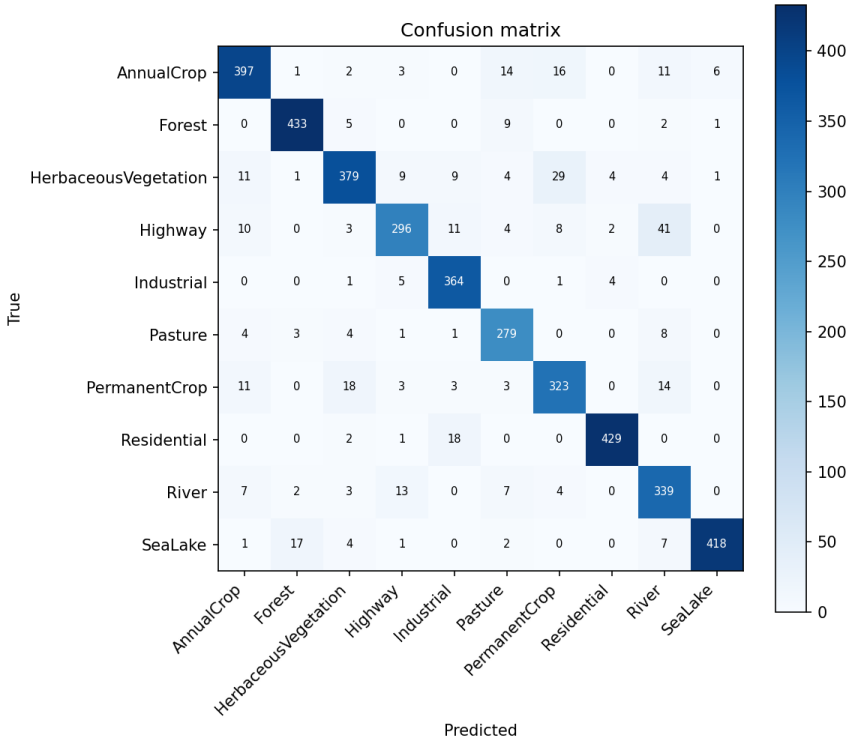


Figure 4: Model A confusion matrix (test set).

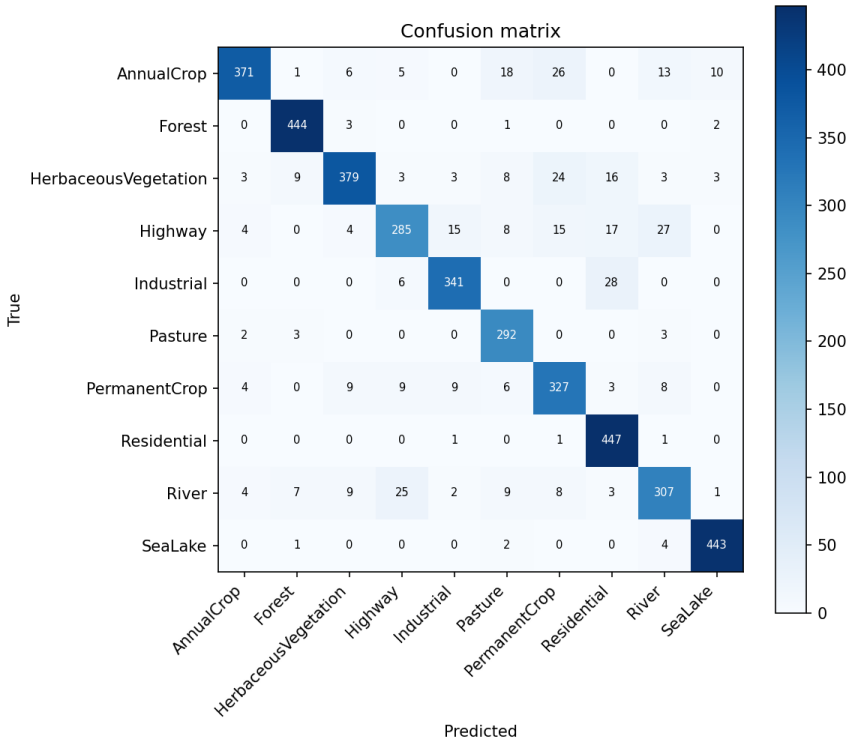


Figure 5: Model B confusion matrix (test set).

Table 7: Per-class precision and recall on the test set, Model A vs. Model B.

	Model A		Model B	
Class	Precision	Recall	Precision	Recall
AnnualCrop	0.900	0.882	0.956	0.824
Forest	0.947	0.962	0.955	0.987
HerbaceousVegetation	0.900	0.840	0.924	0.840
Highway	0.892	0.789	0.856	0.760
Industrial	0.897	0.971	0.919	0.909
Pasture	0.866	0.930	0.849	0.973
PermanentCrop	0.848	0.861	0.815	0.872
Residential	0.977	0.953	0.870	0.993
River	0.796	0.904	0.839	0.819
SeaLake	0.981	0.929	0.965	0.984
Macro average	**0.900**	**0.902**	**0.895**	**0.896**

Both models share the same two hardest confusion pairs: **Highway↔River** (both thin, elongated, linear features at this resolution) and **HerbaceousVegetation ↔ PermanentCrop** (overlapping crop/vegetation texture). These are the lowest-recall classes for both models, which suggests the confusion is a genuine property of the dataset at 64×64 RGB resolution rather than a weakness specific to either architecture.

4.3 Comparison table

Table 8: Model A vs. Model B comparison (20 epochs, Adam, lr = 10^{-3}).

Model	Parameters	Checkpoint size	Median time/epoch	Test accuracy
Model A (standard CNN)	549,290	2,150.1 KB	72.6 s	90.27%
Model B (depthwise-sep.)	13,288	63.4 KB	92.8 s	89.76%

Time/epoch is reported as the median across epochs rather than the mean, since a small number of epochs on this shared CPU-only machine were several times slower than the rest due to background system load; the median is robust to that kind of one-off outlier.

4.4 Discussion: standard vs. depthwise-separable trade-offs

Model B uses only 13,288/549,290 $\approx$ 2.4% of Model A's parameters and produces a checkpoint roughly 34$\times$ smaller (63.4 KB vs. 2,150.1 KB), while losing only 0.51 percentage points of test accuracy (89.76% vs. 90.27%). This is a strong result for the depthwise-separable design: at a fraction of the memory footprint, Model B achieves essentially the same classification quality on this task.

One counter-intuitive result is that Model B's median epoch time (92.8 s) is actually *higher* than Model A's (72.6 s), despite having far fewer parameters and, in theory, fewer multiply-accumulate operations. This is a known practical caveat of depthwise-separable convolutions on general-purpose hardware: grouped/depthwise convolutions have a much lower arithmetic intensity (FLOPs per byte moved) than dense convolutions, so they are more memory-bandwidth-bound and benefit less from the vectorized/cache-friendly BLAS kernels that accelerate dense convolutions on a generic CPU. In practice, this gap is usually closed (or reversed) on inference runtimes and accelerators with kernels specifically optimized for depthwise-separable convolution (e.g. TFLite with XNNPACK, or dedicated mobile NPUs) – the wall-clock disadvantage measured here is a property of this particular unoptimized CPU training environment, not of the operation itself, and should not be read as a general statement about depthwise-separable convolutions being slower in deployment.

5 SOTA Fine-Tuning and Evaluation

Two ImageNet-pretrained, mobile/edge-oriented architectures were fine-tuned end-to-end on the same EuroSAT data splits used for the custom models: **MobileNetV2** and **EfficientNet-B0**. For both, the final ImageNet classification layer was replaced with a fresh Linear($\cdot \to 10$) layer, and *all* layers (not just the new head) were fine-tuned, using Adam with lr = 10^{-4} (a lower learning rate than the custom models, appropriate for not disturbing the pretrained features too aggressively) for 5 epochs.

Pretrained ImageNet features are known to transfer poorly to very small input resolutions, so MobileNetV2 was fine-tuned at an upscaled 128×128 input. EfficientNet-B0 was kept at the native 64×64 resolution purely for computational-cost reasons on the CPU-only hardware available for this assignment: a timing probe showed EfficientNet-B0 at 128×128 would cost approximately 50 minutes per epoch (versus approximately 15 minutes per epoch for MobileNetV2 at the same resolution), which was impractical within the available time budget. This asymmetry is itself a small illustration of the assignment's core theme – compute budget is a real, binding constraint that shapes model and input-resolution choices in edge/resource-constrained settings.

Table 9: SOTA fine-tuning results (5 epochs, Adam, lr $= 10^{-4}$).

Model	Input	Params	Size (MB)	Test accuracy	Macro P / R
MobileNetV2	128×128	2,236,682	8.76	**98.32%**	0.983 / 0.982
EfficientNet-B0	64×64	4,020,358	15.62	97.70%	0.976 / 0.976

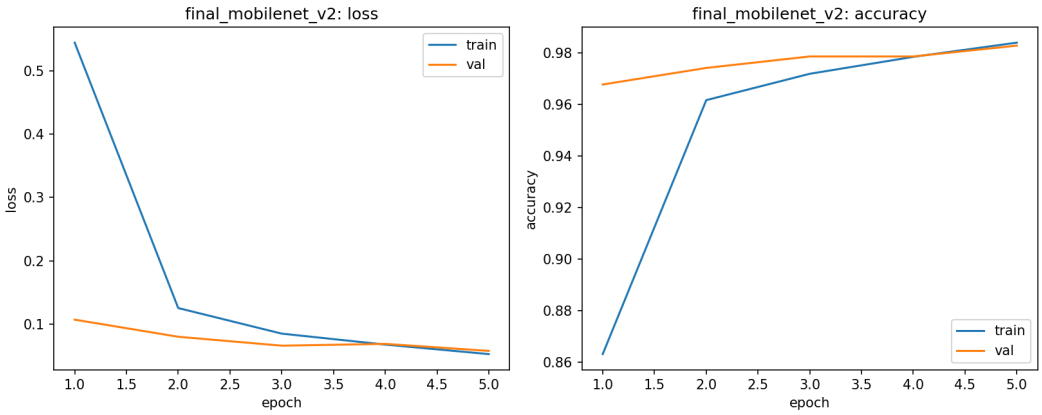


Figure 6: MobileNetV2 fine-tuning: training/validation loss and accuracy.

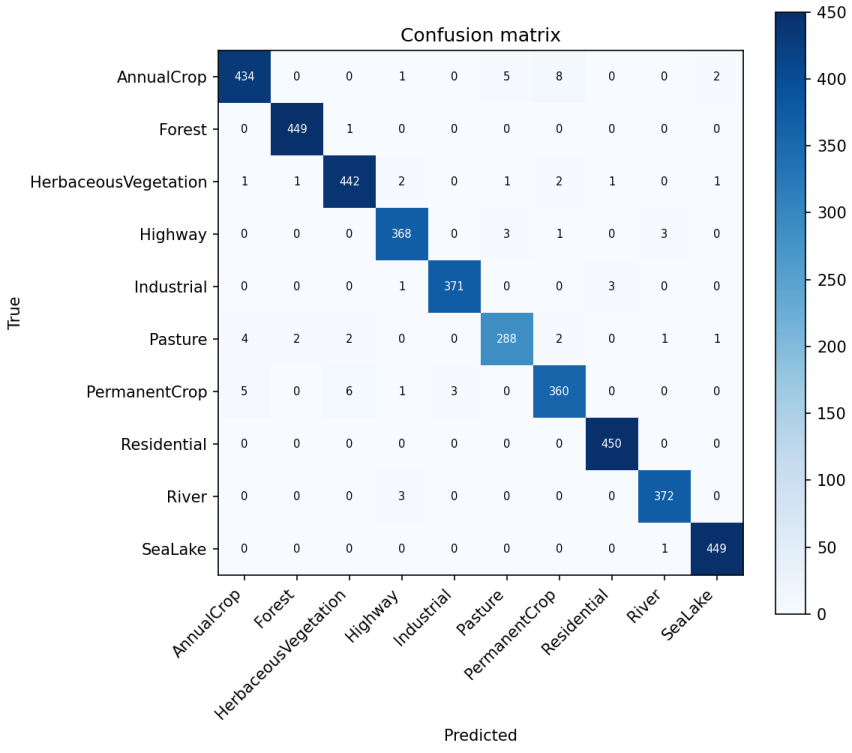


Figure 7: MobileNetV2 confusion matrix (test set).

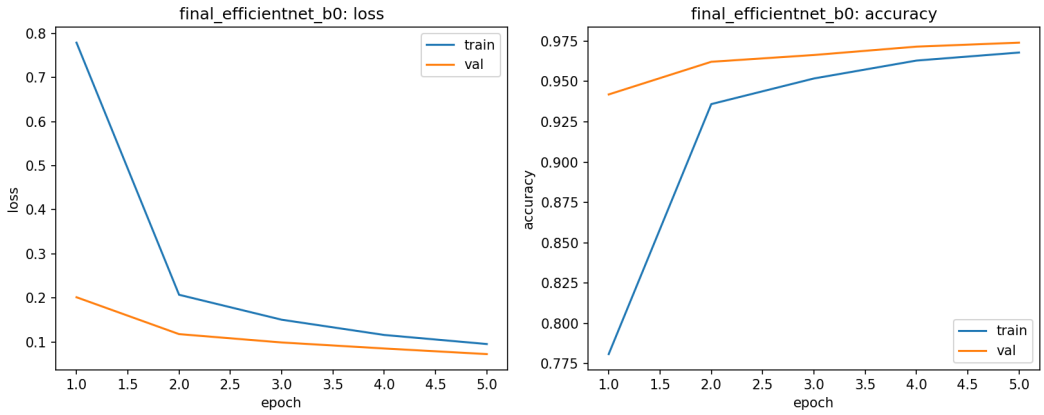


Figure 8: EfficientNet-B0 fine-tuning: training/validation loss and accuracy.

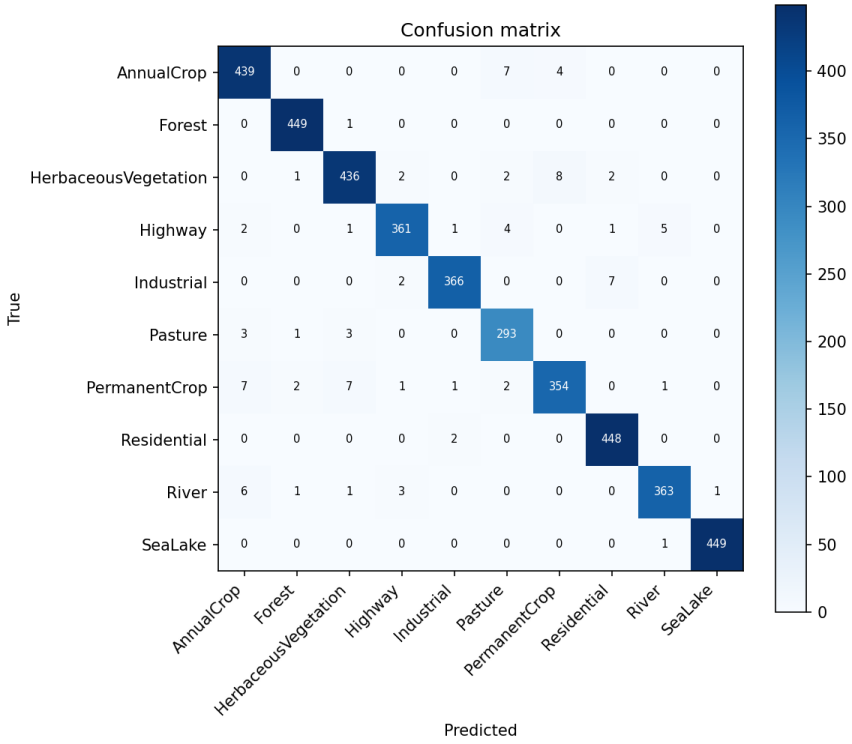


Figure 9: EfficientNet-B0 confusion matrix (test set).

Table 10: Per-class precision and recall on the test set, MobileNetV2 vs. EfficientNet-B0.

	MobileNetV2		EfficientNet-B0	
Class	Precision	Recall	Precision	Recall
AnnualCrop	0.977	0.964	0.961	0.976
Forest	0.993	0.998	0.989	0.998
HerbaceousVegetation	0.980	0.980	0.971	0.967
Highway	0.979	0.981	0.978	0.963
Industrial	0.992	0.989	0.989	0.976
Pasture	0.970	0.960	0.951	0.977
PermanentCrop	0.965	0.960	0.967	0.944
Residential	0.991	1.000	0.978	0.996
River	0.987	0.992	0.981	0.968
SeaLake	0.991	0.998	0.998	0.998
Macro average	**0.983**	**0.982**	**0.976**	**0.976**

Both fine-tuned models comfortably exceed both custom models after only 5 epochs – expected, since they begin from ImageNet-pretrained features rather than random initialization. The same two confusion pairs identified for the custom models (Highway/River, HerbaceousVegetation/PermanentCrop) are still visible in Figures 7 and 9, but at far smaller magnitude, reinforcing that these are genuinely hard classes for this dataset at this resolution rather than an architecture-specific weakness. MobileNetV2 slightly outperforms EfficientNet-B0 here, plausibly helped by its larger 128×128 input resolution.

6 Final Comparison and Trade-off Discussion

Table 11: Model B vs. fine-tuned SOTA models: accuracy, memory, and relative footprint.

Model	Parameters	Size (MB)	Test accuracy	Size vs. Model B
Model B (custom)	13,288	0.062	89.76%	1×
MobileNetV2 (fine-tuned)	2,236,682	8.76	98.32%	$\sim 141\times$
EfficientNet-B0 (fine-tuned)	4,020,358	15.62	97.70%	$\sim 252\times$

Accuracy. The fine-tuned SOTA models outperform Model B by 7.9–8.6 percentage points in test accuracy. This gap is the direct benefit of transfer learning: both SOTA models started from features learned on millions of ImageNet images, while Model B was trained entirely from scratch on EuroSAT's 18,900 training images.

Memory footprint. The gap here runs the other way, and far more steeply: MobileNetV2's checkpoint is about 141× larger than Model B's, and EfficientNet-B0's is about 252× larger. In absolute terms, Model B's 63 KB footprint is small enough to plausibly fit in the flash/RAM of a microcontroller-class device (e.g. typical Cortex-M class chips with hundreds of KB to a few MB of flash), whereas both SOTA models' 8–16 MB footprints exceed that budget by two to three orders of magnitude and would only be viable on richer edge hardware (e.g. a Raspberry Pi, a phone, or a device with a dedicated NPU and MB-scale memory).

Computational cost. Fine-tuning the SOTA models required far fewer epochs to reach a strong result (5 vs. 20 for the custom models), reflecting the sample efficiency of transfer learning. However, each SOTA epoch was substantially more expensive on this CPU-only hardware – MobileNetV2's median epoch time (985 s) was roughly 13× Model B's (93 s), and EfficientNet-B0's (1,258 s) roughly 14×. Total training wall-clock time therefore still favoured Model B for from-scratch training cost, while the SOTA models were more label/sample-efficient per epoch. Inference-time computational cost would need to be assessed separately (not measured in this assignment) and depends heavily on the deployment hardware's support for standard vs. depthwise-separable/grouped convolution kernels.

Recommendation. The choice between a custom sub-100k-parameter model and a fine-tuned SOTA model is fundamentally a question of *where in the accuracy-memory-compute space the target hardware sits*:

- For the tightest microcontroller-class targets (KB-scale memory, no practical way to download or store multi-MB weights), Model B is the only one of the three options that is even feasible, and its ∼90% accuracy may well be an acceptable trade-off for that constraint.
- For richer edge devices with MB-scale memory and reasonable compute (Raspberry Pi-class hardware, phones, edge accelerators), the 8–9-percentage-point accuracy improvement from a fine-tuned SOTA model is substantial and, given the available memory headroom, likely worth the larger footprint – particularly since production inference runtimes (e.g. TFLite/XNNPACK, mobile NPUs) are specifically optimized for depthwise-separable operations and would likely close or reverse the CPU-training-time gap observed in this assignment's unoptimized training environment.
- There is no universally "better" choice among the three models; the right one depends entirely on the deployment target's memory budget and the accuracy requirement of the application.

7 Conclusion

This assignment implemented and evaluated two custom CNNs (a standard 549,290-parameter model and a 13,288-parameter depthwise-separable model, 97.6% smaller) and two fine-tuned pre-trained lightweight models (MobileNetV2 and EfficientNet-B0) on the EuroSAT land-use classification task, with all four models evaluated on an identical 70/15/15 stratified split. The depthwise-separable Model B achieved accuracy within 0.5 percentage points of the much larger Model A at a fraction of the memory footprint, directly demonstrating the value of depthwise-separable convolutions for resource-constrained design. The two fine-tuned SOTA models reached substantially higher accuracy (97.7–98.3%) at the cost of 141–252× more parameters, illustrating the classic accuracy-vs-footprint trade-off that governs edge-deployment model selection: the "best" model depends on where the target hardware sits on that spectrum, not on a single universal accuracy ranking.

References

[1] P. Helber, B. Bischke, A. Dengel, and D. Borth, "EuroSAT: A Novel Dataset and Deep Learning Benchmark for Land Use and Land Cover Classification," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 2019.

[2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *CVPR*, 2018.

[3] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," *ICML*, 2019.

[4] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *ICLR*, 2015.

[5] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.